Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 5

Simulation of Domain Name Resolution using UDP Socket Programming

Name:	Sundareswaran Rajaguru
Register Number:	3122247001048
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	05 August 2026

1. Objective

To develop a client/server application using UDP sockets that simulates a DNS server and DNS client. The client queries the server for domain name resolution and the server responds with the corresponding IP address. The system also simulates client-side caching of recent lookups and handles three categories of errors – non-existent domains (NXDOMAIN), invalid query format, and request timeouts – in order to analyse the suitability of UDP (overhead, latency, reliability) for a DNS-like service.

2. Server Algorithm

1. Maintain a fixed `DNSRecord` table (an array of `domain`→`ip` pairs) acting as the DNS database.
2. Create a UDP socket, bind it to port 8080.
3. Loop forever: block on `recvfrom()` to receive a domain-name query and the client's address.
4. If the query is the special domain `timeout.com`, deliberately `sleep(5)` and send no reply, to simulate a request that never arrives (timeout demonstration).
5. Else if the query does not contain a `'.'` character, it is treated as an *invalid format* and the reply `"ERROR: Invalid Format"` is sent.
6. Otherwise, linearly search the DNS table for the domain:
   - If found, reply with the corresponding IP address.
   - If not found, reply `"NXDOMAIN"`.
7. Send the reply back to the querying client using `sendto()` and loop.

3. Client Algorithm

1. Maintain a local `Cache` array (domain → IP) that starts empty.
2. Create a UDP socket (no `connect()` – connectionless).
3. Repeatedly prompt the user for a domain name (or `exit` to quit).
4. First check the local cache (`searchCache()`):
   - **Cache hit** – print the cached IP immediately, without contacting the server.
   - **Cache miss** – `sendto()` the domain to the server.
5. On a cache miss, wait for a reply using `select()` with a 3-second timeout:
   - If `select()` times out, print `"Request Timed Out!"`.
   - Otherwise `recvfrom()` the reply and interpret it: `NXDOMAIN` ⇒ domain not found; `ERROR: Invalid Format` ⇒ bad query; else the reply is a valid IP, which is displayed and inserted into the client cache.
6. Display the current cache contents after every query and repeat until the user types `exit`.

4. Server Code

4.1 server.c

```c
// server.c

#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define PORT 8080
#define MAX 100
#define RECORDS 5

// Structure for DNS records
struct DNSRecord {
  char domain[50];
  char ip[20];
};

// DNS Database
struct DNSRecord dnsTable[RECORDS] = {{"google.com", "142.250.183.14"},
                                      {"youtube.com", "142.250.66.206"},
                                      {"github.com", "140.82.114.4"},
                                      {"openai.com", "104.18.33.45"},
                                      {"ssn.edu.in", "103.27.235.45"}};

// Checks whether query is in valid domain format
int isValidDomain(char domain[]) { return strchr(domain, '.') != NULL; }

// Searches DNS table
char *resolveDomain(char domain[]) {

  for (int i = 0; i < RECORDS; i++) {

    if (strcmp(domain, dnsTable[i].domain) == 0)
      return dnsTable[i].ip;
  }

  return NULL;
}

// Sends reply to client
void sendReply(int sockfd, struct sockaddr_in client, socklen_t len,
               char message[]) {

  sendto(sockfd, message, strlen(message) + 1, 0, (struct sockaddr *)&client,
         len);
}

// Simulate timeout by intentionally not replying
void simulateTimeout() {

  printf("Simulating Timeout...\n");
  sleep(5);
}

int main() {

  int sockfd;
  char query[MAX];
  char reply[MAX];
```

```
60
61  struct sockaddr_in server, client;
62  socklen_t len = sizeof(client);
63
64  // Create UDP socket
65  sockfd = socket(AF_INET, SOCK_DGRAM, 0);
66
67  server.sin_family = AF_INET;
68  server.sin_port = htons(PORT);
69  server.sin_addr.s_addr = INADDR_ANY;
70
71  bind(sockfd, (struct sockaddr *)&server, sizeof(server));
72
73  printf("====================================\n");
74  printf("     UDP DNS Server Started\n");
75  printf("     Listening on Port %d\n", PORT);
76  printf("====================================\n");
77
78  while (1) {
79
80    memset(query, 0, MAX);
81
82    recvfrom(sockfd, query, MAX, 0, (struct sockaddr *)&client, &len);
83
84    printf("\nReceived Query : %s\n", query);
85
86    // Timeout Simulation
87    if (strcmp(query, "timeout.com") == 0) {
88
89      simulateTimeout();
90      continue;
91    }
92
93    // Invalid Format
94    if (!isValidDomain(query)) {
95
96      strcpy(reply, "ERROR: Invalid Format");
97
98      sendReply(sockfd, client, len, reply);
99
100     printf("Reply Sent : %s\n", reply);
101
102     continue;
103   }
104
105   // DNS Lookup
106   char *ip = resolveDomain(query);
107
108   if (ip != NULL) {
109
110     strcpy(reply, ip);
111
112     printf("Domain Found\n");
113   } else {
114
115     strcpy(reply, "NXDOMAIN");
116
117     printf("Domain Not Found\n");
118   }
119
120   sendReply(sockfd, client, len, reply);
121
122   printf("Reply Sent : %s\n", reply);
```

```
  }

  close(sockfd);

  return 0;
}
```

5. Client Code

5.1 client.c

```
// client.c

#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/select.h>
#include <unistd.h>

#define PORT 8080
#define MAX 100
#define CACHE_SIZE 10

// Structure for Cache
struct Cache {
  char domain[50];
  char ip[20];
};

struct Cache cache[CACHE_SIZE];
int cacheCount = 0;

// Search Cache
char *searchCache(char domain[]) {

  for (int i = 0; i < cacheCount; i++) {

    if (strcmp(domain, cache[i].domain) == 0)
      return cache[i].ip;
  }

  return NULL;
}

// Add entry to Cache
void addCache(char domain[], char ip[]) {

  if (cacheCount >= CACHE_SIZE)
    return;

  strcpy(cache[cacheCount].domain, domain);
  strcpy(cache[cacheCount].ip, ip);

  cacheCount++;
}

// Display Cache
void displayCache() {

  printf("\n----- Client Cache -----\n");

  if (cacheCount == 0) {
```

```
    printf("Cache Empty\n");
    return;
  }

  for (int i = 0; i < cacheCount; i++)
    printf("%s --> %s\n", cache[i].domain, cache[i].ip);

  printf("------------------------\n");
}

int main() {

  int sockfd;
  char domain[MAX], reply[MAX];

  struct sockaddr_in server;
  socklen_t len = sizeof(server);

  sockfd = socket(AF_INET, SOCK_DGRAM, 0);

  server.sin_family = AF_INET;
  server.sin_port = htons(PORT);
  server.sin_addr.s_addr = inet_addr("127.0.0.1");

  printf("=========== DNS Client ===========\n");

  while (1) {

    printf("\nEnter Domain (or exit): ");
    scanf("%s", domain);

    if (strcmp(domain, "exit") == 0)
      break;

    // Check Client Cache
    char *ip = searchCache(domain);

    if (ip != NULL) {

      printf("Cache Hit!\n");
      printf("IP Address : %s\n", ip);
      continue;
    }

    printf("Cache Miss! Querying Server...\n");

    sendto(sockfd, domain, strlen(domain) + 1, 0, (struct sockaddr *)&server,
           len);

    // Timeout using select()
    fd_set readfds;
    struct timeval tv;

    FD_ZERO(&readfds);
    FD_SET(sockfd, &readfds);

    tv.tv_sec = 3;
    tv.tv_usec = 0;

    if (select(sockfd + 1, &readfds, NULL, NULL, &tv) <= 0) {

      printf("Request Timed Out!\n");
      continue;
```

```
    }

    recvfrom(sockfd, reply, MAX, 0, (struct sockaddr *)&server, &len);

    if (strcmp(reply, "NXDOMAIN") == 0) {

      printf("Domain Not Found.\n");
    } else if (strcmp(reply, "ERROR: Invalid Format") == 0) {

      printf("Invalid Domain Format.\n");
    } else {

      printf("IP Address : %s\n", reply);

      addCache(domain, reply);
    }

    displayCache();
  }

  close(sockfd);

  return 0;
}
```

6. Output / Test Cases

The server was started in the background and the client was driven through a single session that exercises every required scenario: a fresh lookup (cache miss), a repeated lookup of the same domain (cache hit), a domain absent from the DNS table (NXDOMAIN), a malformed query (invalid format, no dot), and a query for `timeout.com` (timeout simulation).

6.1 Server Log

```
====================================
     UDP DNS Server Started
     Listening on Port 8080
====================================

Received Query : google.com
Domain Found
Reply Sent : 142.250.183.14

Received Query : notarealdomain.xyz
Domain Not Found
Reply Sent : NXDOMAIN

Received Query : invalidformat
Reply Sent : ERROR: Invalid Format

Received Query : timeout.com
Simulating Timeout...
```

6.2 Client Console Output (all test cases)

```
========== DNS Client ==========

Enter Domain (or exit): Cache Miss! Querying Server...
IP Address : 142.250.183.14
```

```
----- Client Cache -----
google.com --> 142.250.183.14
------------------------

Enter Domain (or exit): Cache Hit!
IP Address : 142.250.183.14

Enter Domain (or exit): Cache Miss! Querying Server...
Domain Not Found.

----- Client Cache -----
google.com --> 142.250.183.14
------------------------

Enter Domain (or exit): Cache Miss! Querying Server...
Invalid Domain Format.

----- Client Cache -----
google.com --> 142.250.183.14
------------------------

Enter Domain (or exit): Cache Miss! Querying Server...
Request Timed Out!

Enter Domain (or exit):
```

Note that the second query for `google.com` produced a *Cache Hit* on the client and therefore never reached the server – this is why the server log shows only one `google.com` entry despite two client-side lookups.

7. Analysis: Suitability of UDP for DNS

- **Communication overhead:** UDP's 8-byte header and connectionless design avoid the three-way handshake and connection-teardown cost of TCP, which matters because a DNS query/response is typically a single small round trip – exactly the workload UDP is optimised for.
- **Latency:** Since no connection setup is required, a UDP query reaches the server and returns a reply in one round trip, minimising latency for the common case, as observed in the near-instantaneous replies for `google.com` and `notarealdomain.xyz`.
- **Reliability:** UDP does not guarantee delivery, so the application must implement its own timeout and retry logic – demonstrated here using `select()` with a 3-second timeout when the server intentionally withholds a reply for `timeout.com`. In production DNS, if a UDP response does not arrive (or exceeds 512 bytes historically), resolvers retry, and can fall back to TCP for larger responses.
- **Caching:** Client-side caching, as implemented here, further reduces the reliance on network round trips (and therefore on UDP's lack of guarantees) for repeated queries, which mirrors how real resolvers use TTL-based caching to cut both latency and server load.

8. Learning Outcomes

- Understood why real-world DNS predominantly uses UDP: low overhead and low latency for small, idempotent query/response exchanges outweigh the lack of built-in reliability.

- Learned to implement application-level timeout handling over UDP using `select()`, since the transport itself provides no delivery guarantee.
- Implemented and observed the effect of client-side caching in reducing redundant network queries.
- Practised handling multiple categories of error conditions (NXDOMAIN, invalid format, timeout) end-to-end across a client-server boundary.
- Reinforced the connectionless `sendto()`/`recvfrom()` programming model and contrasted it with the connection-oriented TCP model used in earlier assignments.